Next-Gen Read Alignment and Contig Query Identification

Research Methods in Biomedical Informatics

Christian Anderson

05 May 2026

CONTENTS

Background

Problem

Imagine you want to confirm that a mutation is present in a mouse model of brittle bone disease. You have millions of short DNA fragments from a sequencer and one short sequence you know should be there. How do you find the full surrounding region?

- Researchers validating transgenic knock-in models need to confirm the sequence around a mutation
- There is no reference to compare against
- All you have is the sequence you are hoping was knocked in

Problem

"Create a program that takes as input the set of all next-generation sequencing reads identified in a sample and an initial query sequence and returns the largest sequence contig that can be constructed from the reads that contains the initial query sequence."

DNA sequencing produces fragments, not full genomes

Genomes can be billions of base pairs long, but sequencing machines can only read hundreds to thousands at a time.

The genome must first be shattered into millions of small fragments, each sequenced independently, then computationally reassembled.

Short reads (Illumina)
150 bp · very accurate
Struggles with repetitive regions

Long reads (PacBio, Oxford Nanopore)
10,000+ bp · less accurate
Spans repetitive regions

Query sequence: ZZZZZZZZZ

Sequencer reads:

ZZZWZYZYY	YYYYYVYYY	XXXXZZZZZ
ZZZZZZZZ	XXXXXXXXX	ZZZZYYYYY
BBBBZBBYB	BBBBZBBYB	DDDDDDDDD

Output contig: XXXXXXXXXZZZZZZZZZYYYYYVYYY

What are kmers?

"In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort."

–*The Hobbit* [1]

What are kmers?

"In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort."
–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

What are kmers?

"In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort."

–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

Simulate Reads by taking random substrings of different length

inahole
intheground
edahobbitno
itnotanastydirty ...

What are kmers?

"In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort."

–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

Simulate Reads by taking random substrings of different length

inahole
intheground
edahobbitno
itnotanastydirty ...

Chop into k length 'mers using a sliding window

$k = 5$
inaho, nahol, nahole, aholei, holein, ... itnot, tnota, notan, otana ...

What are kmers?

"In a hole in the ground there lived a hobbit. Not a nasty, dirty, wet hole, filled with the ends of worms and an oozy smell, nor yet a dry, bare, sandy hole with nothing in it to sit down on or to eat: it was a hobbit-hole, and that means comfort."
–*The Hobbit* [1]

Remove non-alphanumeric characters

inaholeinthegroundtherelivedahobbitnotanastydirty...

Simulate Reads by taking random substrings of different length

inahole
intheground
edahobbitno
itnotanastydirty ...

How do we work backwards?

Chop into k length 'mers using a sliding window

$k = 5$
inaho, nahol, nahole, aholei, holein, ... itnot, tnota, notan, otana ...

What is a graph?

A graph is a structure used to encode connections between things.

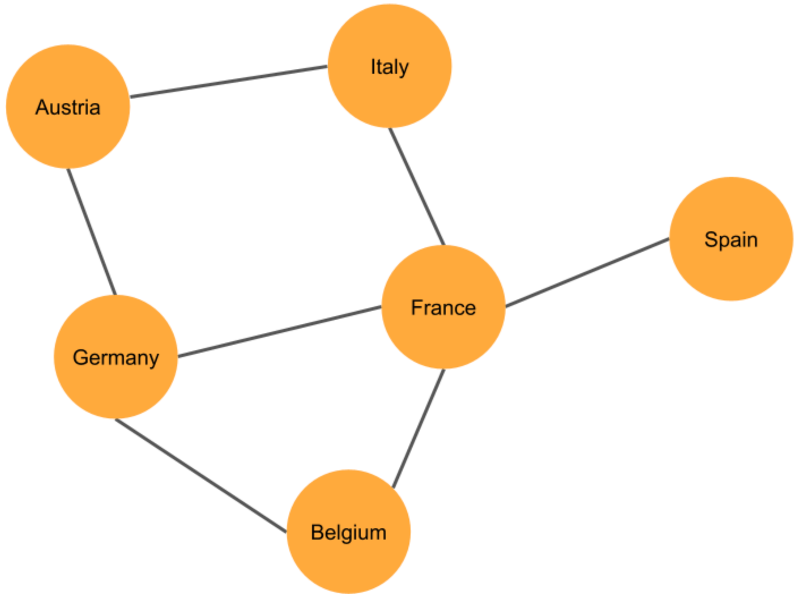

Figure 1: A simple graph used to represent how countries share borders.

De Brujin Graphs

Using the *Hobbit* example, I can plot the graph connecting each kmer to every other kmer that has an overlap.

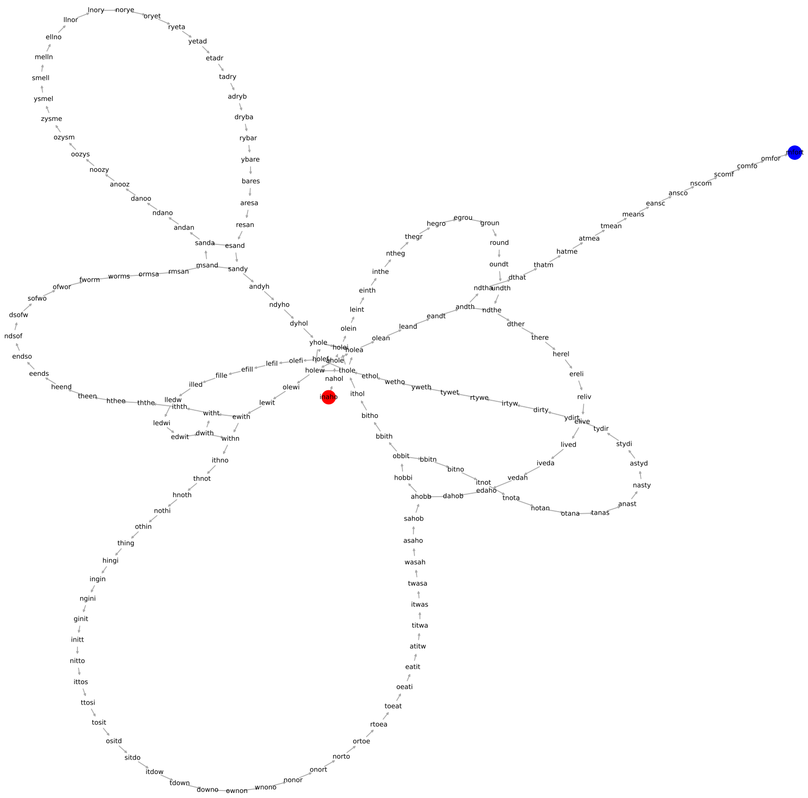

Figure 2: A graph of the 5-length kmers from the first sentence of The Hobbit.

De Brujin Graphs

This kind of graph is composed of kmers with k-1 overlapping characters.
AKA a De Brujin graph:

$k = 5$

```
inaho
 nahol
  ahole
   holei
    olein
     leint
      einth
       inthe ...
inaholeintheground...
```

Following each edge along the graph, the original sentence can be constructed using the last letter of each word.

My Approach

Start from what you know

Instead of exploring the entire graph, anchor on the query and explore outward in both directions.

Genome:

inaholeinthe**ground**therelivedahobbit

Query: **ground** $\rightarrow$ kmers: gro, rou, oun, und

Left (from "gro"): ...ein $\rightarrow$ int $\rightarrow$ nth $\rightarrow$ the $\rightarrow$ heg $\rightarrow$ egr $\rightarrow$ gro
Right (from "und"): und $\rightarrow$ ndt $\rightarrow$ dth $\rightarrow$ the $\rightarrow$ her $\rightarrow$ ere...

Result: ...einthe**ground**there...

Algorithm outline

- Identify kmers that make up the query (*core cloud*)
- Explore left from the leftmost query kmer (DFS)
- Explore right from the rightmost query kmer (DFS)
- Join the longest left + right contigs around the query

Three hyperparameters

k $\rightarrow$ *kmer size*

Too small: same kmer appears everywhere (high coverage, low specificity)

Too large: kmers exceed read length (low coverage, high specificity)

Balance uniqueness vs. frequency

t $\rightarrow$ frequency threshold

Too low: sequencing errors enter the graph as noise

Too high: real but rare kmers are discarded

Balance signal vs. noise

max_visits $\rightarrow$ cycle limit

Prevents infinite loops through repeated sequence

Fixed at 2: allows passing through a node twice before stopping

Handle repeats without endless looping

RESULTS

Validation: three test scenarios

Test 1 **STRT**ABCDEFGABCHIJKDEFENDSTRTABCDEFGABCHIJKDEFEND

- Simple linear sequence with cycle at **DEF**
- Query at the edge of the sequence

Test 2 inaholeinthegroundtherelivedah**obbitnotanast**ydirtywethole...

- Central query
- Explores cycles caused by repeated words like “hobbit” (need for `max_visits` parameter)

Test 3 ATCTGCTGA...

- Simulated DNA reads (A, T, C, G)
- Tests performance with a realistic four-letter alphabet and larger read count

Unit Tests

- Test each function against expected output (integrated into GitHub actions)

Choosing k

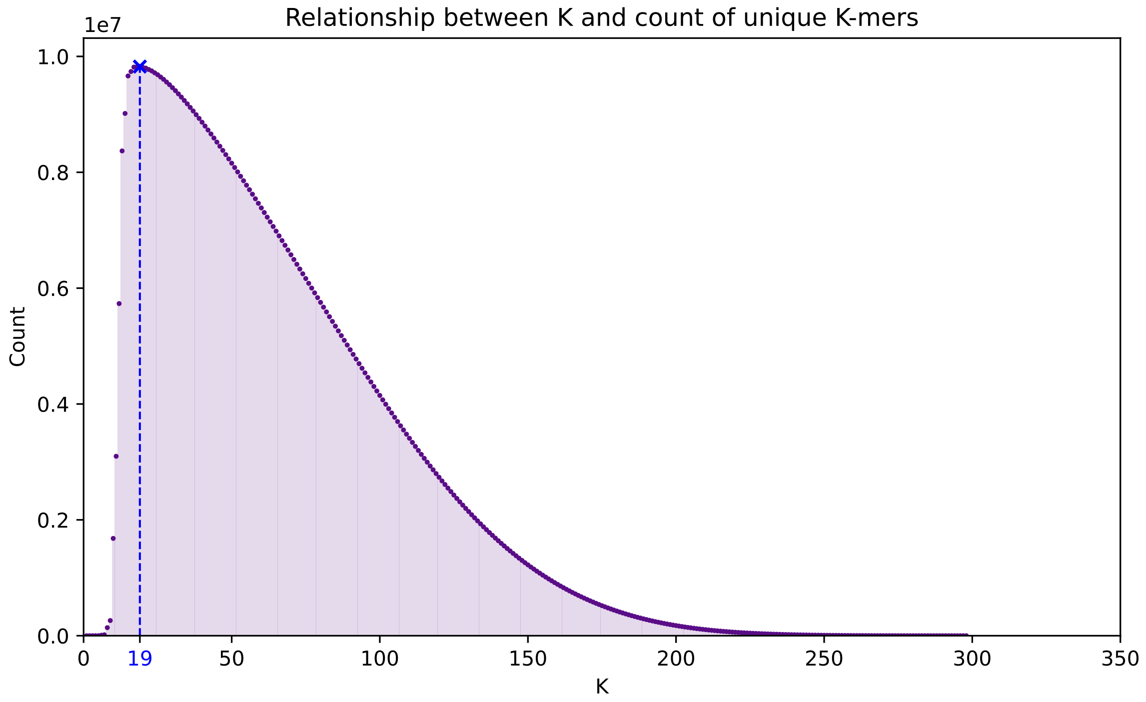

Figure 3: Unique kmers count vs. k

- **Small k (<~10):** nearly every kmer seen repeatedly, very few unique
- **Medium k (~25):** unique count peaks; long enough that most kmers are seen only in one read, short enough to still be fully contained within a read
- **Large k (~>25):** kmers approach read length; count collapses

Chosen value: **k = 19**

Applied to a real sequence

TTCTCCTCTTCCTCATCTCCGGCCTTTCGACCTGCAGCCA
ATAT**GGGATCGGCCATTGAACAAGATGGATTGCACGCAGG**
TTCTCCGGCCGCTTGGGTGGAGAGGCTATTCGGCTATGAC
TGGGCACAACAGACAATCGGCTGCTCTGATGCCGCCGTGT
TCCGGCTGTCAGCGCAGGGGCGCCCGGTTCTTTTTGTCAA
GACCGACCTGTCCGGTGCCCTGAATGAACTGCAGGACGAG
GCAGCGCGGCTATCGTGGCTGGCCACGACGGGCGTTCCTT
GCGCAGCTGTGCTCGACGTTGTCACTGAAGCGGGAAGGGA
CTGGCTGCTATTGGGCGAAGTGCCGGGGCAGGATCTCCTG
TCATCTCACCTTGCTCCTGCCGAGAAAGTATCCATCATGG
CTGATGCAATGCGGCGGCTGCATACGCTTGATCCGGCTAC
CTGCCCATTCGACCACCAAGCGAAACATCGCATCGAGCGA
GCACGTACTCGGATGGAAGCCGGTCTTGTCGATCAGGATG
ATCTGGACGAAGAGCATCAGGGGCTCGCGCCAGCCGAACT
GTTCGCCAGGCTCAAGGCGCGCATGCCCGACGGCGATGAT
CTCGTCGTGACCCATGGCGATGCCTGCTTGCCGAATATCA
TGGTGGAAAATGGCCGCTTTTCTGGATTCATCGACTGTGG
CCGGCTGGGTGTGGCGGACCGCTATCAGGACATAGCGTTG
GCTACCCGTGATATTGCTGAAGAGCTTGGCGGCGAATGGG
CTGACCGCTTCCTCGTGCTTTACGGTATCGCCGCTCCCGA
TTCGCAGCGCATCGCCTTCTATCGCCTTCTTGACGAGTTC
TTCTGAGGGGATCCGCTGGGAGTTCAAAGGAGGGTCCCC
TATCGAATTACAGTGACTGCAGTATATTCTGGAGGATTAG
CTGCTGCACCCTCAGTTTGGGA

BLASTn result:
Mus musculus Col1a2-G610C allele
(type I procollagen alpha2 chain, exon 33)

What this means:
Col1a2 encodes a structural protein in bone. The G610C mutation is a well-characterized mouse model of **osteogenesis imperfecta**, aka, brittle bone disease. This sequence likely derives from a transgenic mouse used to study that condition.

Query sequence shown in purple.

Conclusions

Takeaways

- Anchoring assembly on a known query makes targeted *de novo* assembly tractable
  - exploration starts at most relavent core
- Without ground truth, parameter tuning relies on test performance
- The assembled contig aligns confidently to the *Mus musculus* Col1a2-G610C transgenic allele which is consistent with a mouse model for osteogenesis imperfecta

Limitations & future directions

Current limitations:

- No explicit error correction: the frequency threshold `t` is a blunt proxy
- No ground truth so correctness cannot be confirmed

Future directions:

- Coverage-weighted edges
  - trust high-coverage paths more than low-coverage ones
- Bubble detection to replace reliance on a fixed `max_visits` parameter

References

[1] J. R. R. Tolkien, *The Hobbit*. HarperCollins, 2012.

https://github.com/canderson318/research-methodology-programming-assignment